EE5203 L05 Practice Worksheet

HAL GPIO, pulls, polling, and register evidence

Basri Erdogan

Expected time: 30-45 minutes **Policy:** Attempt every question before consulting the worked solutions. Register questions are paper questions — predict first, verify in the lab.

Part A - Reading HAL configuration

1. For each fragment, state every register field it changes (register name, bit positions, new value):

```
/* (a) PA5 */                          /* (b) PC13 */
config.Pin   = GPIO_PIN_5;             config.Pin  = GPIO_PIN_13;
config.Mode  = GPIO_MODE_OUTPUT_PP;    config.Mode = GPIO_MODE_INPUT;
config.Pull  = GPIO_NOPULL;            config.Pull = GPIO_PULLDOWN;
config.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &config);         HAL_GPIO_Init(GPIOC, &config);
```

2. Write the complete `GPIO_InitTypeDef` block (with `HAL_GPIO_Init`) that reproduces the L04 homework's PB0 output — and state which register fields it writes, at which bit positions.
3. Why does the generated `MX_GPIO_Init` call `HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET)` **before** `HAL_GPIO_Init`? What glitch does this ordering prevent?
4. `{0}` in `GPIO_InitTypeDef config = {0};` — what does it do, and what class of bug appears when it is omitted and `config` is reused for a second pin?

Part B - The two interfaces

5. Give the direct CMSIS equivalent of each HAL call (masks computed):
   a. `HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);`
   b. `HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);`
   c. reading B1: `HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)` — write the `IDR` test that yields the same information.
6. Name one concrete situation where direct register access is the better choice and one where HAL is — with a one-sentence justification each. ("Better" must refer to something checkable: code size, clarity, portability, atomicity, review effort.)
7. A teammate mixes interfaces: CubeMX configures PA5, then their code does `GPIOA->MODER = 0x400;` "to be sure". Name **two** distinct things this plain assignment breaks on this board.

Part C - Inputs and pulls

8. For each wiring, state what the pin reads when the switch is open and when it is closed:
   a. resistor to 3.3 V, switch to ground (B1's wiring);
   b. resistor to ground, switch to 3.3 V;
   c. no resistor, switch to ground.

   Which case is a bug, and what would the program observe?
9. The board's B1 has external R30 to 3.3 V. A student nevertheless configures `GPIO_PULLUP`. Explain what is now physically on the pin, and whether pressed/released readings change.

10. Fill the truth table for the lab's Checkpoint B loop:

B1	`ReadPin` returns	branch taken	LD2
released			
pressed			

Then state the one-token change that inverts the behavior (LED on while pressed).

Part D - Edges, debounce, state

11. This loop is meant to count presses but counts hundreds per press:

```
if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
    ++press_count;
}
```

   a. Why does it over-count even with a perfect, bounce-free button?
   b. Add the `prev`/`cur` edge detection (write the corrected loop body).
   c. With edge detection in place, the count still jumps by 2-3 per press. What causes it, and where does the debounce delay go?

12. Why are `press_count` and `prev` declared `static` inside the loop's enclosing function? What breaks if `static` is removed (assume they are declared inside `while (1)`'s body)?

13. Trace Checkpoint C: `press_count` starts at 0 and B1 is pressed five times. Using `press_count & 0x3U`, which pattern (0-3) is active after each press? Why is `& 0x3U` equivalent to `% 4` here?

Part E - Register evidence

14. After `MX_GPIO_Init()` with the lab's `.ioc` settings, predict the exact values of these fields:

Field	Predicted
`GPIOA->MODER` bits 11:10	
`GPIOA->OTYPER` bit 5	
`GPIOC->MODER` bits 27:26	
`GPIOC->PUPDR` bits 27:26	

15. While you hold B1 down, which single SFR bit changes, in which register — and does `GPIOC->ODR` change too? Why or why not?

16. Your claim: "HAL wrote the same MODER bits I wrote by hand in L04." Describe the two-project comparison (breakpoints, registers, values) that proves it.

Exit self-assessment

Mark one:

- ☐ Ready for the L05 lab without additional review
- ☐ Need to review HAL-name-to-register mapping
- ☐ Need to review pulls and the B1 circuit
- ☐ Need to review edge detection and debounce
- ☐ Need to review the SFR evidence workflow